

Transformation Rule System

Domain Architecture Proposal

A governed semantic foundation for AI-enabled banking
message transformation

Audience: Domain Architecture / Platform Architecture / Technology
Governance

POSITION

Architecture positioning

RTS is a domain architecture capability: a governed transformation-rule knowledge foundation that separates rule truth, runtime projection, indexing, retrieval, and AI consumption.

VALUE

Strategic value

It provides a reusable, auditable and extensible semantic layer for multiple AI-enabled capabilities, instead of creating isolated chatbot-style solutions.

1. Executive architecture summary

RTS should be assessed as a domain architecture pattern, not as an internal knowledge note or a single AI use case. Its purpose is to establish a governed semantic foundation for banking message transformation, so that AI capabilities can operate on approved rule truth rather than raw documents or informal explanations.

THESIS
Core thesis Generic RAG can retrieve text. RTS defines governed rule truth. This distinction matters for banking domains where field-level transformation logic, signoff status, ambiguity and evidence boundaries must remain explicit.
OUTCOME
Architecture outcome A reusable foundation where multiple applications — rule explanation, lineage, impact analysis, release readiness, onboarding readiness and audit explainability — all consume the same governed rule layer.

Recommended framing:

RTS is not another chatbot. It is a governed domain knowledge architecture that enables safe, rule-grounded AI applications across message transformation workflows.

2. Problem statement for architecture

Current transformation knowledge is usually fragmented across code, XSLT, mapping spreadsheets, XML samples, tickets, emails, support notes and Confluence pages. This creates architecture-level risks when AI is introduced directly on top of raw or weakly governed content.

ARCH RISK

Fragmented truth

Different teams rely on different artifacts: code, Excel, documents, old tickets and SME memory. There is no single governed rule object model.

ARCH RISK

Weak explainability

It is hard to explain why a target field is generated, which source path it depends on, and whether the logic has been reviewed or signed off.

ARCH RISK

Unsafe generic AI

A generic chatbot may produce fluent answers without respecting scope, product boundaries, ambiguity, signoff or evidence state.

ARCH RISK

Limited reuse

If every AI use case builds its own prompt and document retrieval layer, the enterprise gets duplicated logic and inconsistent answers.

3. RTS architecture principle

The core architecture principle is truth separation. RTS separates governed truth from runtime retrieval and AI interaction. The LLM is allowed to read, navigate and explain; it is not allowed to invent business logic or promote ambiguity into certainty.

LAYER
Truth Layer Canonical packs define approved rules, lookups, helpers, evidence, review state, ambiguity and signoff boundaries.
LAYER
Projection Layer Only approved runtime objects are projected for AI consumption. Governance noise is not blindly mixed into runtime context.
LAYER
Index Layer Projected resources get stable URIs, metadata, L0/L1/L2 summaries, dependency links and retrieval traces.
LAYER
Query / AI Layer Agents consume the indexed layer through scoped retrieval and structured result shaping; outputs remain drafts for human review where required.

Architecture invariant:

TRS decides what is true. Indexing decides how truth is found. AI explains truth; it does not create truth.

4. Reference architecture

The proposed architecture is intentionally layered so domain governance and AI runtime behavior do not collapse into one uncontrolled surface.

REF
01 Governed source artifacts XML/FpML/SCBML samples, mapping spreadsheets, Java/XSLT logic, lookup tables, defect history, review comments and business clarification.
REF
02 Canonical rule objects rules / lookups / helpers with source, logic, target, dependencies and examples; evidence and review remain explicit governance layers.
REF
03 Runtime projection Approved rule objects are exported as read-only, low-noise runtime resources with stable scope boundaries.
REF
04 Metadata and retrieval index URI, channel, product, pack, object type, dependency graph, L0 recall summary, L1 navigation overview and L2 object content.

REF

05 API-based AI services

Structured prompts and outputs for rule explanation, lineage, impact analysis, readiness assessment and audit narrative generation.

REF

06 Human review and workflow integration

Outputs are reviewed, linked to evidence and integrated into onboarding, release, incident or audit workflows.

5. Use case landscape enabled by RTS

For Domain Architecture, the important point is not any single use case. The important point is that multiple use cases consume the same governed semantic foundation.

FOUNDATION
Foundation capabilities Transformation Rule Explainer; Field Lineage; Source-to-Target Traceability; Rule Dependency Navigation; Evidence-backed Explanation; Ambiguity and Signoff-aware Answering.
WORKFLOW
Delivery & governance workflows Rule-grounded Impact Analysis; Regression Test Recommendation; Release Readiness Review; Rule-grounded Incident Explanation; Change Intent Verification.
BUSINESS
Business-facing applications Client / Upstream Onboarding Accelerator; Product / Flow Readiness Analyzer; Client Impact Intelligence; Regulatory / Audit Explainability; Institutional Knowledge Recovery.
FUTURE
Future expansion Knowledge Decay Detection; Message Quality Radar; On-call Handover; Silent Dependency Discovery; Operational Friction Intelligence.

6. Why this is stronger than isolated AI use cases

Isolated AI use cases are easier to prototype but harder to govern, reuse and scale. RTS turns use cases into consumers of a shared domain foundation.

BENEFIT
Reuse One governed rule foundation supports many applications instead of duplicating prompts, retrieval logic and assumptions per use case.
BENEFIT
Consistency Different applications answer from the same approved rule objects, reducing inconsistent interpretations across teams.
BENEFIT
Traceability Answers can point back to rule objects, dependencies, evidence and review state instead of vague document snippets.
BENEFIT
Governance Ambiguity and signoff state remain first-class architecture concepts, not afterthoughts inside prompts.

BENEFIT

Scalability

New channels, products and packs can follow the same architecture pattern: truth $\rightarrow$ projection $\rightarrow$ index $\rightarrow$ scoped AI consumption.

7. Why Copilot / Yoda are not substitutes

This point should be framed as product-fit, not tool criticism. Copilot and Yoda remain useful, but they do not provide the architecture layer RTS is proposing.

NOT SUBSTITUTE
Copilot Copilot is a personal productivity assistant. It can help users draft, summarize and reason over materials, but it is not a governed transformation-rule truth layer, nor a workflow-embedded backend analysis service.
NOT SUBSTITUTE
Yoda Yoda is a Confluence knowledge-base chatbot. It can answer questions over indexed pages, but RTS needs to reason over rule objects, XML paths, mapping extracts, lookup dependencies, signoff state and operational artifacts beyond Confluence.
REQUIRED
RTS + API token API-based AI allows controlled service integration: structured input, fixed output schema, prompt versioning, validation, evidence references, human review and audit logging.

Architecture distinction:

Copilot and Yoda are user-facing assistance tools. RTS is a governed domain architecture foundation with AI as a controlled consumer of rule truth.

8. Why API token is required

The API token is required because the proposed capability is not manual Q&A.; It is a backend AI capability embedded into domain workflows and controlled by architecture rules.

TOKEN
Workflow integration Trigger analysis from onboarding, UAT, release, incident or audit workflows instead of relying on manual chat sessions.
TOKEN
Structured artifacts Pass XML samples, mapping extracts, rule objects, defect lists and metadata as structured inputs.
TOKEN
Controlled outputs Require JSON / checklist / assessment templates with validation and retry handling.
TOKEN
Auditability Record input sources, prompt version, model version, output, reviewer and evidence links.

TOKEN

Security controls

Support redaction, scope enforcement, access control and human-in-the-loop approval.

TOKEN

Platform extensibility

Expose a reusable AI service layer that multiple RTS-enabled applications can call.

9. Architecture guardrails

These guardrails are important for Domain Architecture approval. They ensure the proposal remains a governed foundation rather than an uncontrolled generative system.

GUARDRAIL
Read-only AI consumption AI reads projected rule truth and operational evidence; it does not modify production rules, mappings, code or configuration.
GUARDRAIL
Human decision boundary AI outputs are analysis drafts, checklists, summaries or recommendations. Final signoff remains with authorized owners.
GUARDRAIL
Scope enforcement Retrieval must first resolve channel, product, pack and object scope to avoid cross-product or cross-system contamination.
GUARDRAIL
Ambiguity preservation Open questions, missing evidence and unsigned rules must remain visible. Unknown is better than wrong.

GUARDRAIL

No runtime learning into truth

Session memory or user feedback cannot silently rewrite canonical rule truth; updates must go through controlled authoring/review/signoff.

GUARDRAIL

Traceable outputs

Material conclusions should reference rule object, dependency, evidence or review state where applicable.

10. Proposed pilot for architecture validation

The pilot should validate the architecture pattern, not merely demonstrate a chatbot. A good pilot proves that governed rule objects can support multiple outputs safely.

PILOT
Pilot scope Use one source-target channel such as Tradition → Stella and a small set of approved packs across COMMON / FXD / FXO.
PILOT
Foundation output Rule explanation, field lineage, dependency navigation and ambiguity-aware answering from projected rule objects.
PILOT
Workflow output Rule-grounded impact analysis and regression test recommendation for a controlled sample change.
PILOT
Business output Onboarding gap checklist or product readiness summary using sample XML and existing rule coverage.

Architecture success criteria

Correct scope resolution, traceable rule references, preserved ambiguity, structured output, human review boundary and repeatable API invocation.

11. Decision points for Domain Architecture

The architecture review should focus on whether RTS is accepted as the semantic foundation pattern for AI-enabled message transformation, and what boundaries are required for safe adoption.

DECISION
1. Approve the layered architecture Truth Layer → Projection Layer → Index Layer → API-based AI consumption.
DECISION
2. Approve RTS as source of governed rule truth Canonical packs remain the court of record for transformation rules.
DECISION
3. Approve API-based AI as controlled consumer AI may explain, summarize, classify and draft outputs, but not decide or mutate truth.
DECISION
4. Approve pilot scope Start with a narrow source-target channel and selected approved packs to validate retrieval and workflow outputs.

DECISION

5. Define ownership

Domain owner for rule truth; platform owner for index/API; workflow owners for consuming applications.

12. Suggested wording for architecture paper

Transformation Rule System is proposed as a governed domain knowledge architecture for banking message transformation. It converts fragmented transformation knowledge into reviewed, signoff-aware and addressable rule objects, then exposes approved runtime projections through indexed retrieval and API-based AI consumption.

The intent is not to build another chatbot. The intent is to establish a reusable semantic foundation on which multiple AI-enabled capabilities can be safely built, including rule explanation, field lineage, impact analysis, regression recommendation, release readiness, onboarding readiness and audit explainability.

Existing tools such as Copilot and Yoda remain useful for user productivity and Confluence knowledge retrieval, but they do not replace a governed rule truth layer, structured runtime projection, scoped retrieval, evidence-linked outputs or workflow-integrated API services.

Control statement:

AI will not make production decisions, approve rules, modify mappings or change system behavior.

AI-generated outputs are structured, evidence-linked drafts for authorized human review.